

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I ATHIN (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: CPU Scheduling – Waiting Time & Turnaround Time Analysis

Given:

Process Burst Time (ms)

P1 10

P2 4

P3 6

P4 2

Since all processes arrive at time 0, we use the Shortest Job First (SJF) Non-Preemptive scheduling algorithm.

Step 1: Execution Order

SJF executes processes in ascending order of burst time:

P4 → P2 → P3 → P1

Gantt Chart

0 2 6 12 22

| P4 | P2 | P3 | P1 |

Waiting Time (WT)

Process WT (ms)

P4 0

P2 2

P3 6

P1 12

Turnaround Time (TAT)

(TAT = WT + BT)

Process TAT (ms)

P4 2

P2 6

P3 12

Process TAT (ms)

P1 22

Average Waiting Time

$$(0 + 2 + 6 + 12) / 4 = 5 \text{ ms}$$

Average Turnaround Time

$$(2 + 6 + 12 + 22) / 4 = 10.5 \text{ ms}$$

Interpretation

The SJF scheduling algorithm selects the process with the shortest burst time first, thereby reducing the average waiting time and turnaround time.

Compared to FCFS, SJF improves system responsiveness and overall CPU performance. Hence, SJF is considered one of the most efficient non-preemptive scheduling algorithms when burst times are known in advance.

Question 2: Round Robin Scheduling – Time Quantum Impact

Given:

Process Burst Time (ms)

P1 20

P2 5

P3 13

P4 8

- **Time Quantum = 4 ms**

Gantt Chart

0-4 P1

4-8 P2

8-12 P3

12-16 P4

16-20 P1

20-21 P2

21-25 P3

25-29 P4

29-33 P1

33-37 P3

37-41 P1

41-42 P3

42-46 P1

46-50 P1

Completion Time

Process CT

P1 50

P2 21

P3 42

P4 29

Turnaround Time

Process TAT

P1 50

P2 21

P3 42

P4 29

Waiting Time

Process WT

P1 30

P2 16

P3 29

P4 21

Analysis of Time Quantum = 10 ms

If the time quantum is increased from 4 ms to 10 ms:

- Number of context switches decreases.
- CPU spends less time switching between processes.
- CPU utilization increases.
- Response time for shorter processes may become worse.
- The behavior of Round Robin becomes similar to FCFS scheduling.

Conclusion

Round Robin scheduling provides fairness among all processes by allowing each process to execute for a fixed time slice. A smaller quantum improves responsiveness but increases context switching, whereas a larger quantum improves CPU efficiency at the cost of response time.

Question 3: Deadlock Safety Check Using Banker's Algorithm

Given:

- **Total Resources:**
 - **A = 10**
 - **B = 5**
 - **C = 7**

Need Matrix Calculation

Need = Maximum – Allocation

Process Need (A,B,C)

P1 (7,4,3)

P2 (1,2,2)

P3 (6,0,0)

P4 (2,1,1)

P5 (5,3,1)

Available Resources

Allocated Resources:

- **A = 7**
- **B = 2**

- $C = 5$

Available:

$(10-7, 5-2, 7-5) = (3,3,2)$

Safety Algorithm Execution

1. P2 can execute.
 - Available becomes (5,3,2)
2. P4 can execute.
 - Available becomes (7,4,3)
3. P5 can execute.
 - Available becomes (7,4,5)
4. P1 can execute.
 - Available becomes (7,5,5)
5. P3 can execute.
 - Available becomes (10,5,7)

Safe Sequence

$P2 \rightarrow P4 \rightarrow P5 \rightarrow P1 \rightarrow P3$

Conclusion

Since all processes can execute successfully and a safe sequence exists, the system is in a safe state. Banker's Algorithm helps avoid deadlocks by ensuring that resource allocation never places the system in an unsafe state.

Question 4: Deadlock Detection – Resource Allocation Graph (RAG)

Given Conditions

- P1 holds R1 and requests R2.
- P2 holds R2 and requests R3.
- P3 holds R3 and requests R1.

Resource Allocation Graph

$R1 \rightarrow P1 \rightarrow R2$

$R2 \rightarrow P2 \rightarrow R3$

$R3 \rightarrow P3 \rightarrow R1$

Cycle Formation

P1 → R2 → P2 → R3 → P3 → R1 → P1

Deadlock Analysis

A deadlock occurs when:

- 1. Mutual Exclusion**
- 2. Hold and Wait**
- 3. No Preemption**
- 4. Circular Wait**

All four conditions are satisfied in this problem. Therefore, the system is deadlocked.

Recommended Strategy

The best solution is Deadlock Prevention by imposing a resource ordering scheme:

R1 < R2 < R3

Processes must request resources in this order only. This removes circular waiting and prevents deadlocks.

Conclusion

The Resource Allocation Graph clearly shows a cycle among the processes and resources. Hence, the system is in a deadlock state, and resource ordering can be used to avoid future deadlocks.

Question 5: CPU Utilization – Multiprogramming Level Analysis

Given:

- Probability of a process waiting for I/O (p) = 0.8**
- Degree of Multiprogramming (n) = 4**

Formula

$$U = 1 - p^n$$

where:

- U = CPU Utilization**
- p = Probability of waiting for I/O**
- n = Number of processes in memory**

(a) CPU Utilization for n = 4

$$U = 1 - (0.8)^4$$

$$U = 1 - 0.4096$$

$$U = 0.5904$$

CPU Utilization = 59.04%

(b) CPU Utilization for n = 6

$$U = 1 - (0.8)^6$$

$$U = 1 - 0.262144$$

$$U = 0.737856$$

CPU Utilization = 73.79%

Comparison Table

Degree of Multiprogramming CPU Utilization

4	59.04%
6	73.79%

Interpretation

Increasing the degree of multiprogramming allows more processes to remain in memory simultaneously. While one process waits for I/O operations, another process can utilize the CPU. As a result, CPU idle time is reduced, leading to improved utilization and system throughput.

Conclusion

The system with six processes achieves better CPU utilization than the system with four processes. Therefore, increasing the degree of multiprogramming enhances overall system performance, provided sufficient memory resources are available.

These versions are detailed enough for university examinations and should comfortably fetch full marks (10 marks each).

Question 6: CPU Scheduling (20 Marks)

(a) FCFS Scheduling Algorithm

In First Come First Serve (FCFS) scheduling, processes are executed in the order of their arrival.

Given:

Process Arrival Time Burst Time

P1	0.0	8
P2	0.4	4
P3	1.0	1

Gantt Chart:

0 8 12 13
| P1 | P2 | P3 |

Turnaround Time (TAT):

- $P1 = 8 - 0 = 8 \text{ ms}$
- $P2 = 12 - 0.4 = 11.6 \text{ ms}$
- $P3 = 13 - 1 = 12 \text{ ms}$

Average Turnaround Time:

- $(8 + 11.6 + 12) / 3 = 10.53 \text{ ms}$

(b) SJF Scheduling Algorithm

In Shortest Job First (SJF), the process with the shortest burst time among the available processes is selected.

At time 0, only P1 is available. Hence, P1 executes first.

Gantt Chart:

0 8 9 13
| P1 | P3 | P2 |

Turnaround Time (TAT):

- $P1 = 8 - 0 = 8 \text{ ms}$
- $P3 = 9 - 1 = 8 \text{ ms}$
- $P2 = 13 - 0.4 = 12.6 \text{ ms}$

Average Turnaround Time:

- $(8 + 8 + 12.6) / 3 = \mathbf{9.53\ ms}$

(c) If Future Arrivals Were Known

If the scheduler knew beforehand that shorter jobs would arrive shortly, it could delay P1 and execute the shorter jobs first.

Possible Schedule:

0-1 Idle

1-2 P3

2-6 P2

6-14 P1

Turnaround Time:

- $P1 = 14\ ms$
- $P2 = 6 - 0.4 = 5.6\ ms$
- $P3 = 2 - 1 = 1\ ms$

Average Turnaround Time:

- $(14 + 5.6 + 1) / 3 = \mathbf{6.87\ ms}$

Conclusion

- FCFS Average TAT = 10.53 ms
- SJF Average TAT = 9.53 ms
- Ideal Scheduler Average TAT = 6.87 ms

Thus, SJF performs better than FCFS. If future arrivals were known in advance, the scheduler could further reduce the average turnaround time. However, operating systems generally do not have knowledge of future process arrivals.

Question 7: CPU Scheduling (20 Marks)

Given Data

Process Burst Time Priority

P1	2	2
P2	1	1

Process Burst Time Priority

P3 8 4

P4 4 2

P5 5 3

(All processes arrive at time 0.)

(a) Gantt Charts

FCFS

0 2 3 11 15 20

|P1|P2| P3 | P4 | P5 |

SJF

0 1 3 7 12 20

|P2|P1|P4| P5 | P3 |

Priority Scheduling

(Higher priority number indicates higher priority.)

0 8 13 15 19 20

| P3 | P5 | P1 | P4 | P2 |

Round Robin (Quantum = 2)

0-2 P1

2-3 P2

3-5 P3

5-7 P4

7-9 P5

9-11 P3

11-13 P4

13-15 P5

15-17 P3

17-18 P5

18-20 P3

(b) Turnaround Time

FCFS

Process Turnaround Time

P1	2 ms
P2	3 ms
P3	11 ms
P4	15 ms
P5	20 ms

SJF

Process Turnaround Time

P2	1 ms
P1	3 ms
P4	7 ms
P5	12 ms
P3	20 ms

Priority Scheduling

Process Turnaround Time

P3	8 ms
P5	13 ms
P1	15 ms
P4	19 ms
P2	20 ms

(c) Waiting Time

FCFS

Process Waiting Time

P1	0 ms
P2	2 ms
P3	3 ms
P4	11 ms
P5	15 ms

Average Waiting Time = **6.2 ms**

SJF

Process Waiting Time

P2	0 ms
P1	1 ms
P4	3 ms
P5	7 ms
P3	12 ms

Average Waiting Time = **4.6 ms**

Priority Scheduling

Process Waiting Time

P3	0 ms
P5	8 ms
P1	13 ms
P4	15 ms
P2	19 ms

Average Waiting Time = **11 ms**

(d) Algorithm with Minimum Average Waiting Time

Algorithm Average Waiting Time

FCFS 6.2 ms

SJF 4.6 ms

Priority 11 ms

Conclusion

Among all the scheduling algorithms, Shortest Job First (SJF) produces the minimum average waiting time. Therefore, SJF is considered the most efficient scheduling algorithm for this set of processes because it minimizes process waiting time and improves overall system performance.

Question 8: CPU Scheduling (10 Marks)

An airport has five check-in counters that serve passengers arriving at different times. Each passenger is treated as a process with varying service times. During peak hours, passengers requiring special assistance are assigned higher priority.

FCFS Scheduling

The First Come First Serve (FCFS) scheduling algorithm serves passengers in the order in which they arrive. It is simple to implement and ensures fairness among passengers. However, during peak hours, passengers requiring immediate assistance may experience longer waiting times.

Priority Scheduling

Priority Scheduling assigns higher priority to passengers requiring special assistance, such as elderly individuals, disabled passengers, or VIP travelers. These passengers are served before others, reducing their waiting time and improving service quality.

Comparison of FCFS and Priority Scheduling

Parameter	FCFS	Priority Scheduling
Fairness	High	Moderate
Implementation	Simple	Moderate
Waiting Time	Higher	Lower for priority users
Response Time	Moderate	Faster
Special Assistance Support	No	Yes

Parameter	FCFS	Priority Scheduling
Peak Hour Efficiency	Moderate	High

Analysis

- FCFS provides equal treatment to all passengers.
- Priority Scheduling improves service efficiency for passengers with urgent needs.
- Priority Scheduling reduces turnaround time for high-priority passengers.
- A combination of FCFS and Priority Scheduling can provide better results in real-world airport systems.

Conclusion

Priority Scheduling is better suited for airport check-in systems during peak hours because it provides faster service to passengers who require immediate attention. However, FCFS remains useful due to its simplicity and fairness. Therefore, a hybrid scheduling approach can be adopted to achieve both efficiency and fairness in airport operations.

RUBRIC S FOR CALCULATION: Numerical Problems

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation